



Lab 08 - Polymorphism

Requirements

Name your solution: **Week08Polymorphism**

1. Working with shapes

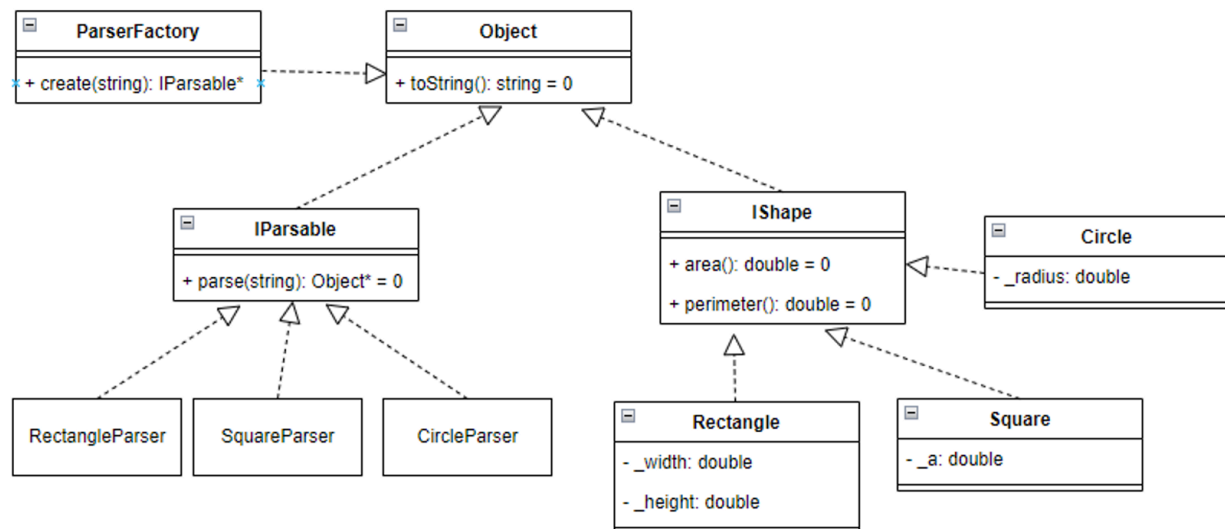
- Name your project: **Shapes**
- Given the below main function, write all necessary files to make it run.

```
int main() { vector<string> lines = { "Square: a=12", "Circle: r=12",  
"Square: a=7", "Rectangle: w=3, h=4", "Rectangle: w=6, h=8", "Circle: r=5",  
"Square: a=8" }; ParserFactory factory; vector<IShape*> shapes; for (auto&  
line : lines) { // Example: line = "Square: a=12" vector<string> tokens =  
Utils::String::split(line, ": "); IParsable* parser =  
factory.create(tokens[0]); // "Square"=> SquareParser IShape* shape =  
dynamic_cast<IShape*> parser->parse(tokens[1]); // "a=12" => Square(_a = 12)  
shapes.push_back(shape); } for (auto& shape : shapes) { // Polymorphism cout  
<< shape->toString() << ": area=" << shape->area() << ", perimeter=" <<  
shape->perimeter() << endl; } }
```

Sample output

Square a=12: area=144, perimeter=48 Circle r=12: area=432, perimeter=72
 Square a=7: area=49, perimeter=28 Rectangle w=3, h=4: area=12, perimeter=14
 Rectangle w=6, h=8: area=48, perimeter=28 Circle r=5: area=75, perimeter=30
 Square a=8: area=64, perimeter=32

Hints



```

namespace Utils {
    class String {
    public:
        static vector<string> split(string haystack, string needle) {
            vector<string> result;
            int startPos = 0;
            size_t foundPos = haystack.find(needle, startPos);
            while (foundPos != string::npos) {
                int count = foundPos - startPos;
                string token = haystack.substr(startPos, count);
                result.push_back(token);
                startPos = foundPos + needle.length();
                foundPos = haystack.find(needle, startPos);
            } // The remaining string token =
            haystack.substr(startPos, haystack.length() - startPos);
            result.push_back(token);
            return result;
        }
    };
}
  
```

Usage of ParserFactory

```
ParserFactory factory; string name = "Rectangle"; IParsable* parser =
factory.create(name); // return new RectangleParser name = "Circle"; parser =
factory.create("Circle"); // return new CircleParser name = "Square"; parser
= factory.create("Square"); // return new SquareParser
```

So basically, inside the **create** function is just a switch

```
IParsable* ParserFactory::create(string type) { IParsable* parser = NULL; if
("Rectangle" == type) { parser = new RectangleParser(); } else if ("Square"
== type) { parser = new SquareParser(); } else if ("Circle" == type) { parser
= new CircleParser(); } return parser; }
```

Improvement **01** to make the code shorter and easier to read

```
IParsable* ParserFactory::create(string type) { map<string, IParsable*>
container = { {"Rectangle", new RectangleParser()}, {"Square", new
SquareParser()}, {"Circle", new CircleParser()} }; IParsable* result =
container [type]; return parser; }
```

Improvement **02** to make the code becomes more upgradable

```
// Let use dependency injection => inversion of control class ParserFactory {
private: map<string, IParsable*> _container; // A container for all the
dependencies injection public: void registerWith(IParsable* parser) { // Do
the injection here _container.insert( {parser->parsedObjectName(), parser} );
} IParsable* create(string type) { return _container[type]; // Ask container
to resolve the needed type } } int main() { // In the beginning of our
application ParserFactory factory; // We inject what we want to work with
here factory.registerWith(new RectangleParser()); factory.registerWith(new
SquareParser()); factory.registerWith(new CircleParser()); //... }
```

2. Working with employees

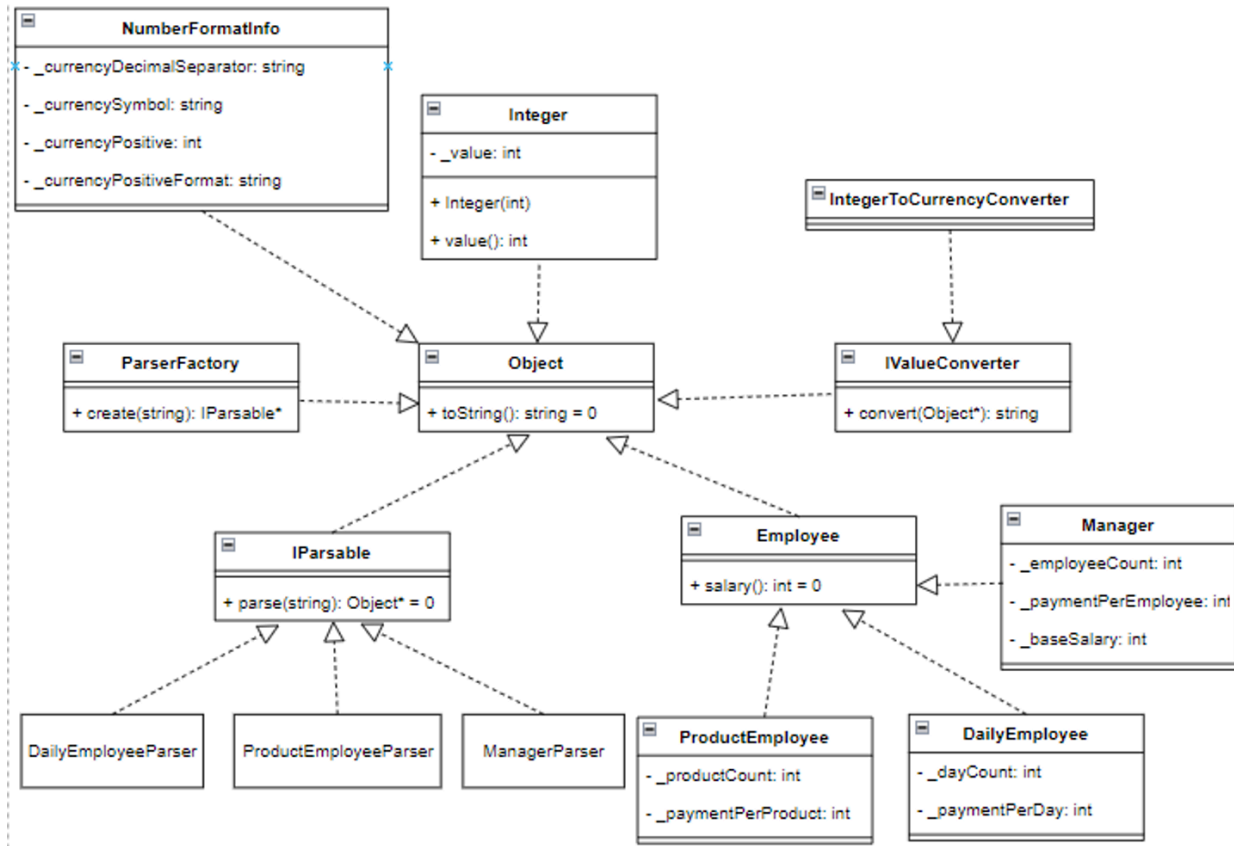
- Name your project: **EmployeesPolymorphism**
- Given the below main function, write all necessary files to make it run.

```
int main() { ParserFactory factory; factory.registerWith(new
DailyEmployeeParser()); factory.registerWith(new ProductEmployeeParser());
factory.registerWith(new ManagerParser()); vector<string> lines = {
"ProductEmployee productCount=790, paymentPerProduct=20000", "DailyEmployee
dayCount=20, paymentPerPay=1000000", "ProductEmployee productCount=810,
paymentPerProduct=19000", "ProductEmployee productCount=900,
paymentPerProduct=18000", "Manager employeeCount=49,
paymentPerEmployee=500000, baseSalary=6000000", "DailyEmployee dayCount=22,
paymentPerPay=1100000", "Manager employeeCount=37, paymentPerEmployee=500000,
baseSalary=7000000", "DailyEmployee dayCount=18, paymentPerPay=1500000",
"Manager employeeCount=30, paymentPerEmployee=500000, baseSalary=5000000" };
vector<Employee*> employees; for (string line : lines) { // "ProductEmployee
productCount=790, paymentPerProduct=20000" int splitPosition =
line.find_first_of(" "); string type = line.substr(0, splitPosition); // type
= "ProductEmployee" IParsable* parser = factory.create(type); // data =
"productCount=790, paymentPerProduct=20000", string data =
line.substr(splitPosition + 1, line.length() - splitPosition - 1); Employee*
employee = dynamic_cast<Employee*>(parser->parse(data));
employees.push_back(employee); } IntegerToCurrencyConverter converter; for
(auto& employee : employees) { Integer* value = new Integer(employee-
>salary()); cout << employee->toString() << ", salary:" <<
converter.convert(value) << endl; } }
```

Sample output

```
Product employee, salary:15.800.000 d Daily employee, salary:20.000.000 d
Product employee, salary:15.390.000 d Product employee, salary:16.200.000 d
Manager, salary:25.000.000 d Daily employee, salary:24.200.000 d Manager,
salary:19.000.000 d Daily employee, salary:27.000.000 d Manager,
salary:15.500.000 d
```

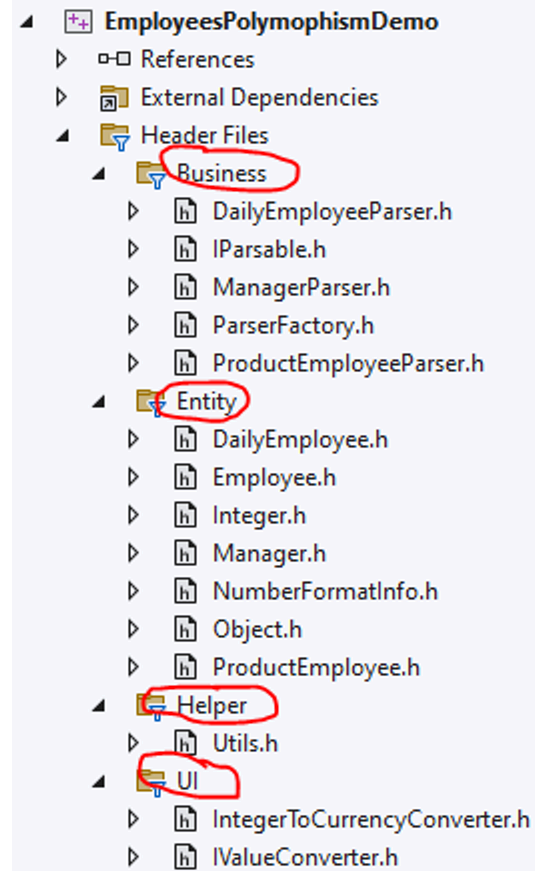
Hints



```

class NumberFormatInfo: public Object
{
public:
    string _currencyDecimalSeparator = ".";
    string _currencySymbol = "d"; //
    https://stackoverflow.com/questions/11111111/determine-currency-symbol-position-from-culture
    int _currencyPositive = 3;
    string _currencyPositiveFormat = "n";
    "n$" "$ n" "n $" public: string
    currencyDecimalSeparator(); string
    currencySymbol(); int currencyPositive();
    currencyPositiveFormat(); public: string
    toString(); };

```



```

class ParserFactory: public Object { private: map<string, IParsable*>
_container; public: void registerWith(IParsable* parser); IParsable*
create(string type); string toString(); }; void
ParserFactory::registerWith(IParsable* parser) { _container.insert({ parser-
>parsedObjectName(), parser }); } IParsable* ParserFactory::create(string
type) { return _container[type]; }

```

```

string IntegerToCurrencyConverter::convert(Object* object) { auto number =
dynamic_cast<Integer*>(object); int value = number->value(); stringstream
builder; NumberFormatInfo info; int count = 0; while (value != 0) { int
lastDigit = value % 10; builder << lastDigit; count++; if (count == 3) {
builder << info.currencyDecimalSeparator(); count = 0; } value /= 10; }
string numberSeparated = builder.str(); // "000.000.1"
reverse(numberSeparated.begin(), numberSeparated.end()); string format =
info.currencyPositiveFormat(); "n $" ==> "1.000.000 d" string symbol =
info.currencySymbol(); // "d" //
https://stackoverflow.com/questions/17459450/how-to-determine-currency-
symbol-position-for-a-culture string result = regex_replace(format,
regex("n"), numberSeparated); result = regex_replace(result, regex("\\$"),
symbol); return result; }

```

```

class Integer: public Object { private: int _value; public: Integer();
Integer(int); int value(); string toString(); };

```